

NSF SpectrumX Data and Algorithm Competition (SpX-DAC)

Final Competition Report

Team Name: EDGE-RAN Gary / gunnchOS3k MLV

Edmund Gunn Jr. – Team Lead, Systems Integration and 6G / AI-RAN Framing

Ananya Jha – Machine Learning Modeling Lead

Noah Newman – Evaluation, Visualization, and Tooling Lead

Institutional Affiliations:

New York University Tandon School of Engineering

University of Massachusetts

Alma College

Team Lead Contact: eg3573@nyu.edu

March 2026

This report accompanies the final SpX-DAC algorithm submission. The main body is limited to the core competition detector and its evaluation; pseudocode, supplementary ablations, and the Gary digital-twin research extension are deferred to the appendix.

Executive Summary

This report addresses the 2025–2026 NSF SpectrumX Data and Algorithm Competition (SpX-DAC) as a binary spectrum-occupancy detection problem: given a 1-second raw I/Q measurement, determine whether it contains an active user transmission or only noise-like background activity. The task was approached as a practical engineering problem under constrained packaging, runtime, and reproducibility requirements. The resulting design philosophy was deliberately pragmatic: establish a dependable submission path with simple classical baselines, then build toward a compact learned detector that could be trained locally on the official labeled subset and deployed through the required `evaluate(filename)` interface without heavy dependencies or runtime training.

The initial submission used spectral flatness with an energy-detection fallback, providing a safe baseline, verifying leaderboard compatibility, and ensuring fail-safe execution. Building on that foundation, the final detector uses a compact feature pipeline that extracts time-domain power statistics, PSD-based structure measures, and short-lag autocorrelation summaries from each I/Q sample. These features support lightweight supervised models, including logistic-regression, linear-SVM, and gradient-boosting variants, with operating-point selection used to balance true detections against false alarms.

A major practical objective was strict separation between official competition data and public-facing tooling. Official SDS data were kept local only; no raw competition `.npz` files were committed to the repository, deployed in Streamlit, or included in leaderboard archives. The strongest public operating point was a prior-corrected gradient-boosting model (v9) that achieved 0.85 public leaderboard accuracy at 797.54s total leaderboard runtime. The core contribution is therefore a reproducible baseline-to-learned-model workflow, a compact deployment wrapper, and a competitive accuracy–efficiency operating point that remains explicit about what was actually submitted.

1 Problem Description

The SpectrumX Data and Algorithm Competition asks participants to detect active user transmissions from raw I/Q measurements. In operational terms, the task is a binary decision problem: given a 1-second capture, determine whether it contains a structured user transmission or only noise-like background activity. According to the official competition framing, the measurements come from real field data collected during the NSF SpectrumX July 2025 VLA experiment, targeting 5G-like signals in the lower 7 GHz band.

The dataset was interpreted as labeled waveform captures whose occupancy status must be inferred directly from low-level signal behavior rather than from decoded packets, protocol symbols, or precomputed images. For supervised development, the labeled subset contained 240 one-second files, with 160 occupied and 80 unoccupied examples, so the learning problem is both binary and class-imbalanced. The detection objective is to produce a dependable occupied-versus-unoccupied decision from each raw I/Q sample while remaining compatible with the competition’s packaging and runtime constraints.

2 Dataset Usage and Preprocessing

Official competition data were accessed locally through the SpectrumX Data System workflow and Python package. Throughout development, the official SDS data remained on local machines; only code and compact learned artifacts were used in the public leaderboard package. This separation kept the submission minimal, reproducible, and compliant with competition data-handling expectations.

2.1 Dataset use for training, validation, and testing

The competition release contains both labeled and unlabeled material. Supervised model development used a local labeled subset constructed from files associated with official ground truth. This subset contains 240 files: 160 occupied samples and 80 unoccupied samples, giving a 2:1 class imbalance. These labeled files were used for training and local validation through balanced 5-fold stratified cross-validation during model selection and threshold tuning. The 200 unlabeled files in `trainingData/` were not used for supervised training. Final testing was performed only through the hidden leaderboard evaluation.

2.2 Input handling and normalization

The detector accepts several common `.numpy` layouts, including complex arrays of shape $(N,)$ and real-valued arrays of shape $(N,2)$ interpreted as $[I,Q]$. All supported inputs are converted into a common 1D `complex64` representation before downstream processing. Each waveform is capped at a fixed maximum I/Q length to control runtime and memory use.

2.3 Preprocessing and feature extraction

Preprocessing is intentionally lightweight: (1) normalize the input format to a common complex I/Q representation, (2) cap the sample length, (3) compute Welch PSD statistics and time-domain summaries, (4) standardize features using statistics from the labeled training subset, and (5) sanitize numerical edge cases with finite-value replacement. The final feature family combines time-domain power and amplitude statistics, PSD-shape features, frequency-localization measures, and short-lag autocorrelation summaries. PSD statistics are computed with Welch’s method. No spectrogram-image pipeline or explicit learned front-end was used in the final submission.

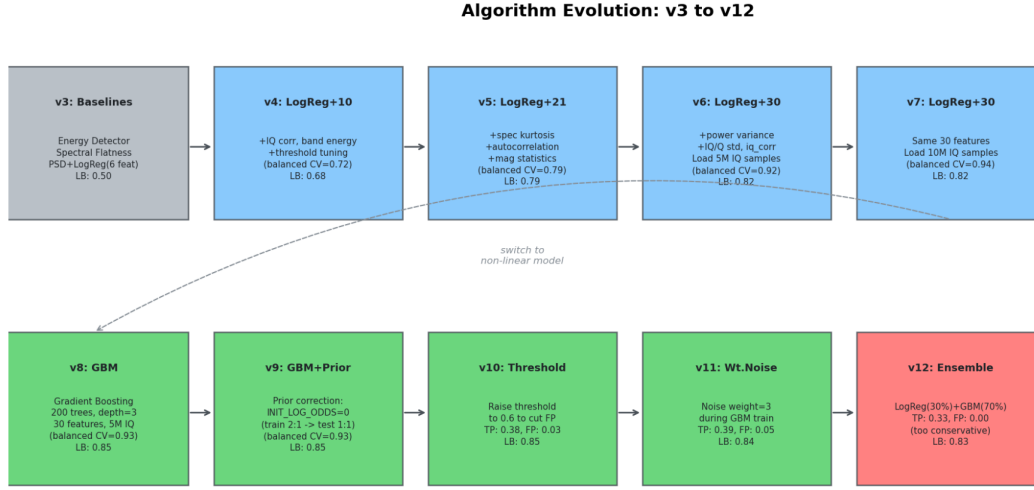


Figure 1: Algorithm evolution from baseline detectors through logistic-regression variants to the gradient-boosting and ensemble family.

3 Algorithm Design

The final detector evolved through three stages: classical heuristic baselines, compact handcrafted feature extraction, and lightweight supervised models. Figure 1 summarizes the progression across submission versions.

At a high level, each raw `.npy` waveform is converted into a common complex I/Q representation, reduced to a compact feature vector, scored by a trained classifier, and mapped to a final binary decision by thresholding. This structure was chosen to balance three constraints at once: competitive detection performance, small exported artifacts, and reliable execution inside the required `evaluate(filename)` interface.

3.1 Classical baselines

The initial submission path used two simple signal-processing detectors. The energy detector computes mean waveform power and compares it against a threshold. Although fast and interpretable, it is limited because structured transmissions and noise can overlap in energy under different propagation and interference conditions. To improve robustness while remaining lightweight, a spectral-flatness detector was added. Spectral flatness compares the geometric mean of the PSD to its arithmetic mean: noise-like waveforms tend to exhibit flatter spectra, while structured transmissions often produce more concentrated spectral content. In the initial leaderboard wrapper, spectral flatness served as the primary decision rule and energy detection served as a fallback path.

3.2 Feature engineering and lightweight models

After the submission path was validated, the detector moved from direct heuristic decisions to a compact feature-based pipeline. The central idea was to preserve the interpretability of classical signal processing while allowing a learned model to combine multiple weak cues. Rather than use a large neural network, the final design intentionally relied on a small handcrafted feature family that is inexpensive to compute, robust to input-layout differences, and easy to export in a minimal submission artifact.

The normalized waveform is summarized through three groups of descriptors: time-domain statistics (mean power, amplitude variability, crest factor, kurtosis, and related dispersion measures), frequency-domain and PSD summaries (spectral flatness, spectral entropy, PSD peak statistics, band-energy ratios, centroid, rolloff, and percentile-based PSD descriptors), and short-lag autocorrelation features intended

to capture temporal structure that is less likely to appear in purely noise-like samples. Logistic regression and linear SVM established that compact handcrafted features could support meaningful discrimination, but leaderboard accuracy saturated, indicating that a purely linear decision surface was insufficient.

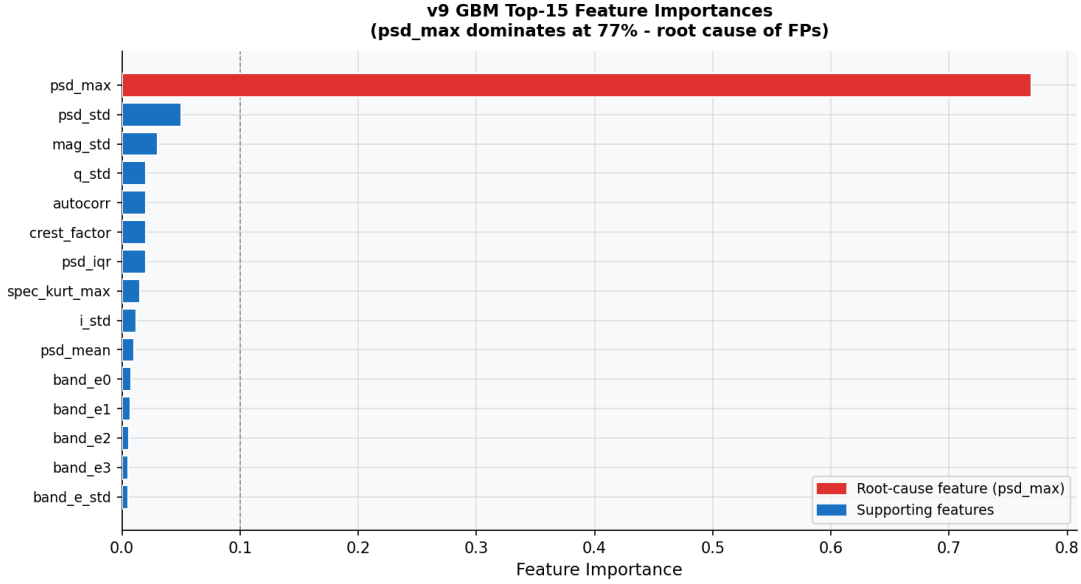


Figure 2: Feature-importance profile for the v9 gradient-boosting model. The dominance of `psd_max` motivated later anti-false-positive feature engineering.

Gradient boosting preserved the same compact 30-feature representation used in the strongest linear models while replacing the linear decision rule with a more flexible nonlinear classifier. Figure 2 shows that the feature `psd_max` dominated the learned importance profile in the v9 gradient-boosting model, which helped explain why some leaderboard false positives appeared to be driven by strong localized spectral peaks.

3.3 Training procedure and decision rule

Training loads the labeled local subset, extracts the shared feature vector for each waveform, standardizes features where appropriate, and performs balanced 5-fold stratified cross-validation for model comparison. Candidate models were compared under both local balanced-validation behavior and leaderboard behavior. Operating-point selection was treated as a first-class design choice because the labeled local subset has a 2:1 occupied-to-unoccupied class ratio while the hidden leaderboard set appears approximately balanced. The strongest public operating point, v9, uses the same 30-feature gradient-boosting model as v8 but corrects the class-prior mismatch between the local training distribution and the hidden leaderboard distribution. Concretely, v9 adjusts the initial log-odds prior from the training ratio toward a balanced test prior without changing the learned tree structure.

The leaderboard submission requires a `main.py` exposing `evaluate(filename)` and returning 0 or 1. The submission wrapper was therefore designed to remain operational even if the learned artifact could not be loaded: it robustly loads the I/Q file, extracts the shared feature vector, evaluates the trained artifact when present, falls back to spectral flatness if needed, then to energy detection, and returns a safe default 0 on unexpected failure. This fail-safe structure keeps the deployment surface auditable and robust.

Table 1: Local and leaderboard-side timing comparison across the main submission versions. Local per-file timing is reported when available from saved model metadata. Leaderboard timing is the total runtime reported by the public evaluation server on the hidden batch.

Ver.	Family	Feat.	Local/file (s)	LB total (s)	Notes
v4	LogReg	10	–	541.95	Welch $n_{\text{perseg}} = 8192$
v5	LogReg	21	–	555.33	added spectral-kurtosis segments
v6	LogReg	30	–	882.26	richer PSD processing
v7	LogReg	30	2.632	1134.89	10M I/Q cap
v8	GBM	30	1.435	–	local only
v9	GBM	30	1.435	797.54	best public result
v10	GBM	30	1.435	1513.66	slower leaderboard run
v11	GBM	30	1.941	895.23	saved-metadata timing
v12	Ensemble	30	1.646	1528.78	ensemble runtime penalty
v14	GBM	33	1.858	pending	local smoke test only

4 Implementation, Computational Complexity, and Efficiency

The implementation is partitioned into three layers: shared feature utilities for robust I/Q loading, PSD computation, and handcrafted feature extraction; local experiment scripts that access the official dataset through the SDS workflow, build the labeled subset manifest, train candidate models, and validate exported artifacts; and the leaderboard inference wrapper exposing `evaluate(filename)` and returning a plain Python integer for each 1-second I/Q file. This separation keeps the competition package minimal while still enabling reproducible local experimentation.

Signal processing and machine learning are connected through a shared feature interface. Raw `.npz` I/Q inputs are normalized into a common complex representation, after which the signal-processing layer computes waveform summaries including PSD statistics, time-domain moments, and short-lag autocorrelation features. These descriptors form a fixed-length vector consumed by the learned model. The same feature extractor therefore supports classical baselines, linear models, and gradient boosting without changing the leaderboard interface. The inference wrapper is designed to degrade gracefully: if the learned artifact is unavailable or malformed, the wrapper falls back to the spectral-flatness detector, then to energy detection, and finally to a safe default 0 if an unexpected exception occurs.

The local development stack uses Python together with `numpy==2.3.5`, `pandas==2.3.3`, `scipy==1.16.3`, `scikit-learn==1.7.2`, and `spectrumx==0.1.17`. The SpectrumX SDK is used locally for SDS dataset access only. The dominant per-sample cost lies in the front-end signal-processing path: file I/O, waveform normalization, Welch PSD estimation, and handcrafted feature extraction. Time-domain statistics and short-lag autocorrelation features are linear or near-linear in waveform length, while PSD computation is driven by FFT-based segment processing. In practice, feature extraction dominates runtime; once the feature vector is available, the learned decision rule is inexpensive. Memory use is similarly modest because the submission processes one input sample at a time and the exported artifact stores only a compact model representation plus feature and calibration metadata.

Table 1 shows that the gradient-boosting family achieved roughly 1.4–1.9 s per file on the primary local timing machine, with v9 matching v8 because v9 changes only prior correction and thresholding, not the feature-extraction path. On the public leaderboard, v9 required 797.54 s over the hidden 240-file batch, or approximately 3.32 s per file if divided naively. These numbers are reported separately because they were collected on different execution surfaces. Training and most local timing experiments were performed on Ananya Jha’s Windows 11 laptop with an AMD Family 25 Model 124 CPU (12 logical cores), 15.3 GB RAM, and Python 3.13.9; development, report writing, and Streamlit integration were also performed on Edmund Gunn Jr.’s Apple M2 system with 8 GB RAM, macOS 26.3, and Python 3.11.

Table 2: Local cross-validation comparison for the supervised feature-based models. Versions **v4–v14** are directly comparable because they use balanced 5-fold stratified cross-validation on the 240 labeled files. Here FP is false alarm rate and FN is miss detection rate on the local validation protocol.

Ver.	Family	Feat.	Bal. Acc.	FA	MD	TP	TN	Thr.	Notes
v4	LogReg	10	0.7167	0.25	0.07	0.43	0.25	0.2907	+IQ features
v5	LogReg	21	0.7942	0.01	0.19	0.31	0.49	0.4872	richer PSD+IQ
v6	LogReg	30	0.9188	0.03	0.15	0.35	0.47	0.2994	full 30 features
v7	LogReg	30	0.9375	0.02	0.15	0.35	0.48	0.3110	10M I/Q cap
v8	GBM	30	0.9313	–	–	–	–	0.0281	not submitted alone
v9	GBM	30	0.9313	0.04	0.11	0.39	0.46	0.5000	prior-corrected
v10	GBM	30	0.9313	0.03	0.12	0.38	0.47	0.6000	conservative threshold
v11	GBM	30	0.9313	0.05	0.11	0.39	0.45	0.8733	noise-weighted GBM
v12	Ensemble	30	0.9375	0.00	0.17	0.33	0.50	0.7542	too conservative
v14	GBM	33	0.9313	0.01	0.06	0.44	0.49	0.9861	anti-FP; LB pending

5 Experimental Results

5.1 Local validation on the labeled training set

The directly comparable local experiments are the balanced 5-fold stratified cross-validation runs on the 240 labeled files. Earlier **v3** baselines used limited hold-out evaluation and are therefore treated only as setup validation rather than part of the main supervised comparison.

Table 2 highlights three patterns. First, richer handcrafted features materially improved local discrimination, with balanced accuracy rising from 0.7167 in **v4** to 0.9375 in **v7**. Second, once the 30-feature representation was in place, the move from logistic regression to gradient boosting changed the problem from one of raw separability to one of operating-point control. Third, the anti-false-positive engineering pass in **v14** reduced the local false alarm rate to 0.01 and the miss detection rate to 0.06, although its public leaderboard result was still pending at the time of writing.

5.2 Public leaderboard behavior

At final revision, the best public leaderboard result was submission **v9**, which ranked 14th out of 131 visible entries with 0.85 accuracy and 797.54s execution time. This result confirms that the compact feature-based detector materially improved over the initial heuristic submission path and remained competitive without relying on a large or computationally heavy model.

Several public entries achieved slightly higher accuracy, reaching the 0.88–0.89 range, but often at substantially higher runtime. Version **v9** is therefore interpreted not as the highest visible public score, but as the strongest deployment-oriented operating point in this submission family: a model that balances predictive quality, robustness, and execution efficiency more effectively than several slower alternatives.

5.3 Error patterns and interpretation

The early baseline detector was intentionally conservative and tended to suppress false alarms at the cost of weaker true-positive detection. The feature-based models improved over that baseline by combining multiple complementary cues rather than relying on a single heuristic statistic. The remaining errors appear consistent with weak or low-SNR structured transmissions, hidden-domain mismatch between local validation and leaderboard conditions, and imperfect threshold calibration under class-prior shift. The **v9** prior-correction step materially improved public behavior without changing the underlying tree structure, suggesting that calibration and operating-point selection were at least as important as raw separability once the 30-feature representation had matured.

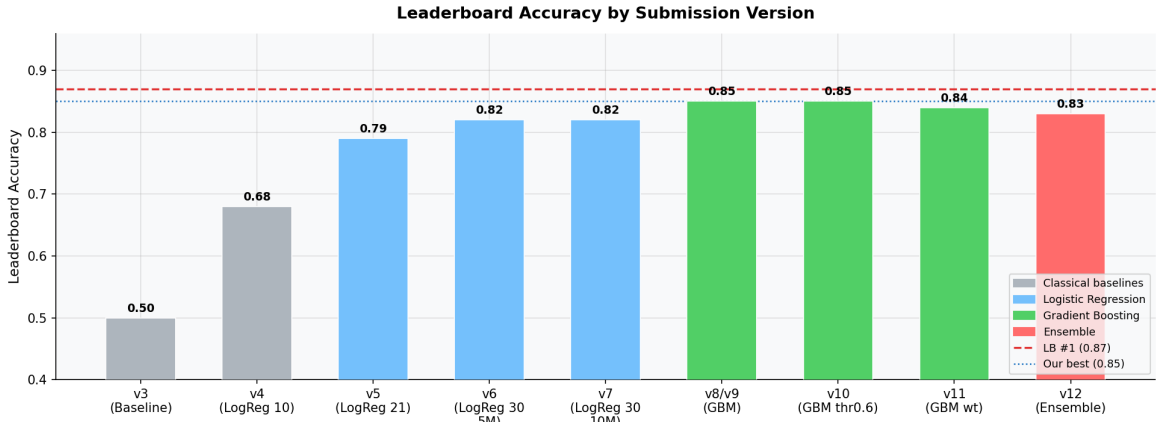


Figure 3: Leaderboard accuracy progression across submission versions. The transition from classical baselines to logistic-regression variants and finally to the gradient-boosting family delivered the strongest public operating point.

6 Discussion

6.1 Strengths

The main strengths of the final detector are efficiency, robustness, interpretability, and a favorable deployment profile. The submission wrapper uses compact handcrafted feature extraction together with a small exported model artifact, which keeps inference costs low and reduces the likelihood of leaderboard-side execution failures. The wrapper also degrades gracefully: if the learned artifact is unavailable or malformed, it falls back to simpler baseline detectors rather than failing outright. Because the detector is built on waveform, spectral, and autocorrelation summaries rather than a large end-to-end model, its behavior remains relatively interpretable. This was particularly useful in understanding later false-positive behavior, where feature-importance analysis helped guide prior correction and anti-false-positive engineering.

6.2 Limitations

The best public model did not achieve the highest visible leaderboard accuracy, indicating that there is still room to improve the underlying discrimination power of the detector. The current feature-based pipeline may miss higher-order waveform structure that a richer model class could exploit. A second limitation is dependence on the size and diversity of the officially labeled subset: because the detector is trained only on that labeled local subset, residual sensitivity to low-SNR signals, ambiguous spectral structure, and hidden-domain mismatch is difficult to remove fully. A third limitation is threshold sensitivity: reducing false alarms too aggressively can suppress true positives, while a looser operating point can improve recall at the cost of more false alarms.

6.3 Design tradeoffs and improvements

The central tradeoff is straightforward: some public competitors appear to gain a few points of accuracy by spending materially more runtime and complexity. The present design instead prioritizes a balanced operating point built from compact features, small exported artifacts, and a robust evaluation wrapper. This choice lowers execution cost and packaging risk while still delivering competitive public performance. Immediate next steps include stronger feature-selection and ablation studies, better score calibration, more systematic threshold selection under class-prior shift, semi-supervised use of the unlabeled portion of the official dataset, anomaly-detection fusion for ambiguous cases, and more targeted anti-false-positive feature engineering. The Gary digital-twin extension should be understood as a future testbed for synthetic stress testing rather than as a redesign of the submitted detector.

7 Reproducibility Checklist

Environment. Python 3.13.9 (Anaconda) on the primary training/timing machine and Python 3.11 on the macOS integration machine; Windows 11 on the primary machine and macOS 26.3 on the secondary machine; AMD Family 25 Model 124 CPU (12 logical cores), 15.3 GB RAM on the primary machine; Apple M2 with 8 GB RAM on the secondary machine.

Libraries. `numpy==2.3.5`, `pandas==2.3.3`, `scipy==1.16.3`, `scikit-learn==1.7.2`, and `spectrumx==0.1.17`.

Official data handling. Official data were downloaded locally through the SpectrumX SDS workflow; authentication was handled through `SDS_SECRET_TOKEN` or a local `.env` file; no raw official competition `.npz` files were committed or uploaded publicly.

Run instructions.

- `python3 scripts/download_official_spxdac.py -out data/competition_dataset`
- `python3 scripts/build_official_labeled_subset.py -dataset-root data/competition_dataset -out-dir data/competition_subset`
- `python3 scripts/train_feature_detector.py -iq-dir data/competition_subset/iq -metadata data/competition_subset/metadata.csv -file-col file -label-col label -out submissions/leaderboard`
- `PYTHONPATH=. python3 scripts/test_leaderboard_submission.py`
- `PYTHONPATH=. python3 scripts/validate_feature_leaderboard_submission.py`

Input/output. Input: path to a `.npz` file containing a 1-second I/Q sample. Output: integer 0 or 1 returned by `evaluate(filename)`.

Expected runtime. For v9, the local mean runtime was approximately 1.435s per file for feature extraction plus inference on the primary timing machine, measured over the 240 labeled files. The public leaderboard runtime was 797.54s over the hidden 240-file evaluation batch, or approximately 3.32s per file if divided naively. Repeated local timing on the same machine may vary by roughly $\pm 5\text{--}15\%$ because of OS scheduling and laptop thermal effects.

Randomness. The final model family used balanced 5-fold stratified cross-validation on the 240 labeled files. Exact integer random seeds and explicit `random_state` values were not preserved in the final exported summary tables, so the workflow is reproducible at the pipeline level but not guaranteed to be bit-for-bit deterministic unless those seed values are recovered from the original training scripts or saved model metadata.

References

- [1] NSF SpectrumX, “Student Data & Algorithm Competition,” official competition page, accessed Apr. 2026.
- [2] SpectrumX Data & Algorithm Competition, “Final Competition Report Requirements,” 2026.
- [3] P. D. Welch, “The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms,” *IEEE Transactions on Audio and Electroacoustics*, vol. 15, no. 2, pp. 70–73, 1967.
- [4] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] P. Virtanen *et al.*, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.
- [6] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [7] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.
- [8] SpectrumX Data System (SDS), “`spectrumx` Python SDK / gateway documentation,” accessed Apr. 2026.
- [9] Streamlit, “Streamlit documentation,” accessed Apr. 2026.
- [10] EDGE-RAN Gary / gunnchOS3k MLV, “`spectrumx-ai-ran-gary`,” GitHub repository, 2026.
- [11] R. Sun, D. Viorel, W. Keusgen, and R. Gebremedhin, “Empirical Path Loss Model and Small-Scale Fading Statistics in an Indoor Office Environment in 6 and 37 GHz Shared Bands,” in *Proc. EuCAP*, 2024.
- [12] R. Sun, W. Keusgen, and D. Viorel, “MIMO Channel Capacity Measurements in an Indoor-Office Environment at 6 and 37 GHz,” in *Proc. IEEE GLOBECOM Workshops*, 2024.
- [13] A. Alkhateeb, “DeepMIMO: A Generic Deep Learning Dataset for Millimeter Wave and Massive MIMO Applications,” arXiv:1902.06435, 2019.
- [14] J. Hoydis *et al.*, “Sionna: An Open-Source Library for Next-Generation Physical Layer Research,” arXiv:2203.11854, 2022.
- [15] J. Hoydis *et al.*, “Sionna RT: Differentiable Ray Tracing for Radio Propagation Modeling,” arXiv:2303.11103, 2023.
- [16] F. Aït Aoudia *et al.*, “Sionna RT: Technical Report,” arXiv:2504.21719, 2025.
- [17] NVIDIA, “Aerial Omniverse Digital Twin,” documentation, 2026.
- [18] O-RAN Software Community, “O-RAN Architecture Overview,” documentation, accessed Apr. 2026.
- [19] AI-RAN Alliance, “AI-RAN Alliance official site,” accessed Apr. 2026.
- [20] S. R. Khosravirad, I. van de Voorde, H. Viswanathan, M. Moisio, and R. Koblitz, *Digital Twins in 6G Networks: Revolutionizing Connectivity and RAN Performance*. Nokia Bell Labs white paper, 2025.

- [21] T. Li, H. Lei, H. Guo, M. Yin, Y. Hu, Q. Zhu, and S. Rangan, “Digital Twin-Enhanced Wireless Indoor Navigation: Achieving Efficient Environment Sensing With Zero-Shot Reinforcement Learning,” *IEEE Open Journal of the Communications Society*, vol. 6, pp. 2356–2372, 2025.
- [22] T. S. Rappaport, Y. Xing, O. Kanhere, S. Ju, A. Madanayake, S. Mandal, A. Alkhateeb, and G. C. Trichopoulos, “Wireless Communications and Applications Above 100 GHz: Opportunities and Challenges for 6G and Beyond,” *IEEE Access*, vol. 7, pp. 78729–78757, 2019.
- [23] R. W. Heath Jr. and A. Lozano, *Foundations of MIMO Communication*. Cambridge University Press, 2018.
- [24] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson/Prentice Hall, 2010.
- [25] A. V. Oppenheim, A. S. Willsky, and S. H. Nawab, *Signals and Systems*, 2nd ed. Prentice Hall, 1997.
- [26] A. Papoulis and S. U. Pillai, *Probability, Random Variables and Stochastic Processes*, 4th ed. McGraw-Hill, 2001.
- [27] S. Rangan, “ECE-GY 6023 / Introduction to Wireless Communications,” course materials and GitHub repository, New York University.

A Algorithm Pseudocode

Algorithm 1: Lightweight feature-based occupancy detection

Input: path to 1-second I/Q file

Output: binary prediction $\hat{y} \in \{0, 1\}$

1. If the file does not exist, return 0.
2. Load the waveform with a robust I/Q loader that accepts common `.npy` layouts.
3. If the waveform is malformed, empty, or cannot be converted to a valid complex I/Q representation, return 0.
4. Normalize the waveform to a common complex64 representation and cap it to the maximum allowed I/Q length.
5. Extract the shared feature vector:
 - (a) time-domain power and amplitude statistics,
 - (b) Welch PSD summaries and frequency-shape features,
 - (c) short-lag autocorrelation summaries.
6. If a trained model artifact is present and valid:
 - (a) align features to the artifact’s expected feature order;
 - (b) apply stored normalization or standardization parameters if required by the model;
 - (c) evaluate the model score or posterior using the stored model family and parameters;
 - (d) apply any stored prior correction or calibration step;
 - (e) compare the resulting score to the stored decision threshold;
 - (f) return 1 if the score exceeds threshold, else return 0.
7. Otherwise, compute the spectral-flatness baseline and return its binary decision if successful.
8. If spectral flatness fails, compute the energy-detector baseline and return its binary decision if successful.

9. If any unexpected exception occurs at any stage, return the safe default 0.

Algorithm 2: Local supervised training and model-selection workflow

Input: labeled local I/Q directory, metadata CSV

Output: compact trained artifact for inference

1. Load the metadata CSV defining labeled files and binary occupancy labels.
2. For each labeled file:
 - (a) load and normalize the waveform;
 - (b) extract the shared feature vector;
 - (c) append features and label to the local design matrix.
3. Stack all feature vectors into X and labels into y .
4. Define candidate model families, including logistic regression, linear SVM, and gradient boosting.
5. For each candidate model:
 - (a) run balanced stratified 5-fold cross-validation on the labeled subset;
 - (b) record validation metrics and error behavior;
 - (c) sweep decision thresholds on validation outputs;
 - (d) select an operating point that balances true detections against false positives.
6. Compare candidate models using both local validation behavior and practical deployment constraints.
7. Select the preferred model family and retrain it on the full labeled subset.
8. If needed, apply prior correction to account for mismatch between the local class ratio and the expected evaluation-time class ratio.
9. Export a compact artifact containing:
 - (a) model family and parameters,
 - (b) feature names and ordering,
 - (c) feature normalization statistics,
 - (d) threshold and any calibration or prior-correction metadata.

B Supplementary Ablations

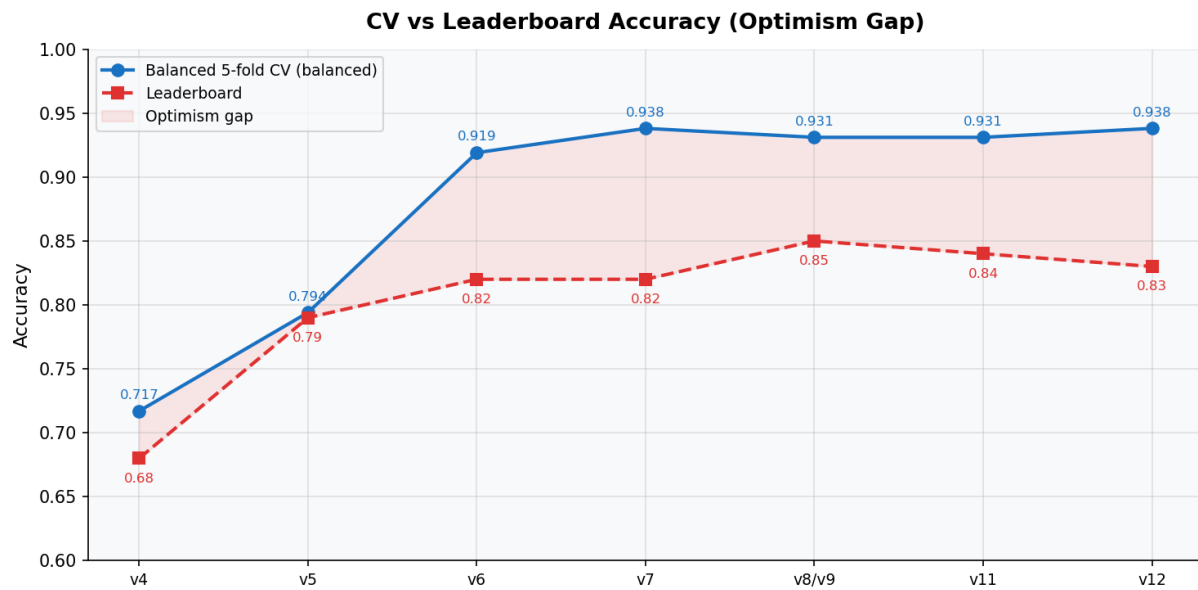


Figure 4: Cross-validation versus public leaderboard accuracy across major submission versions.

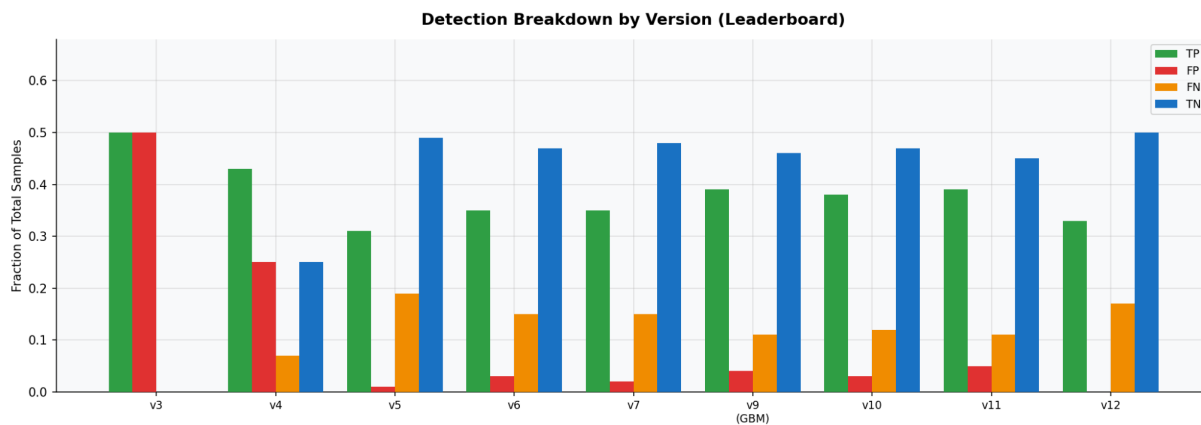


Figure 5: Leaderboard detection breakdown by submission version.

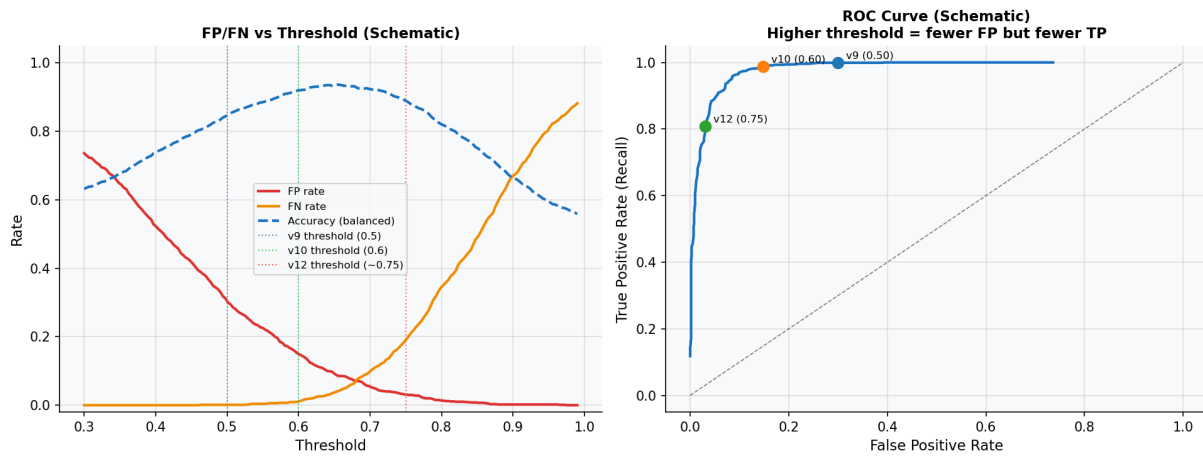


Figure 6: Threshold tradeoff analysis illustrating FP/FN movement under different operating points.

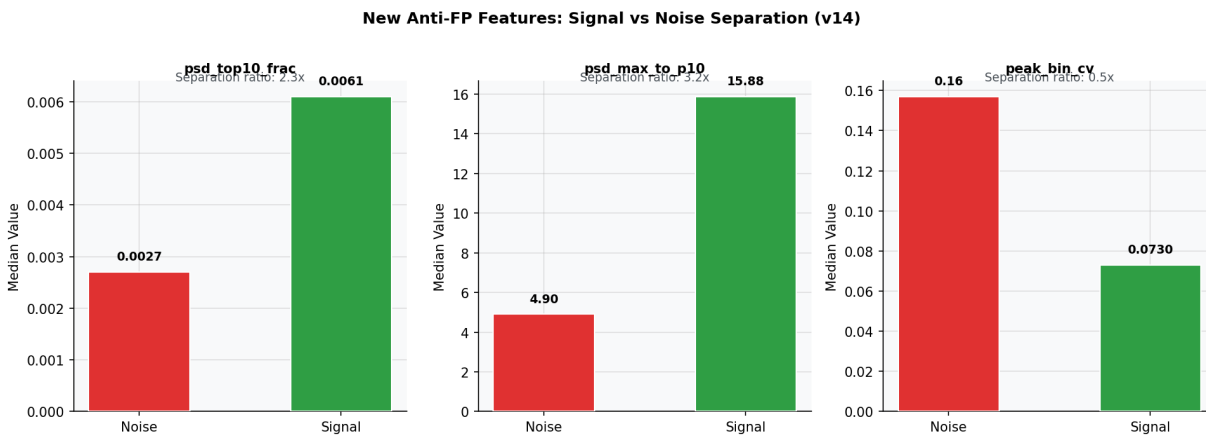


Figure 7: Supplementary anti-false-positive feature analysis for the v14 engineering pass.

C Gary Digital-Twin Research Extension

The Gary digital-twin extension was developed alongside the competition detector but is not the basis of the submitted leaderboard algorithm. We include it in the appendix as supplementary technical material so that the main paper remains centered on the judged detector while still documenting the broader systems context of the project.

C.1 Streamlit extension overview and scenario walkthrough

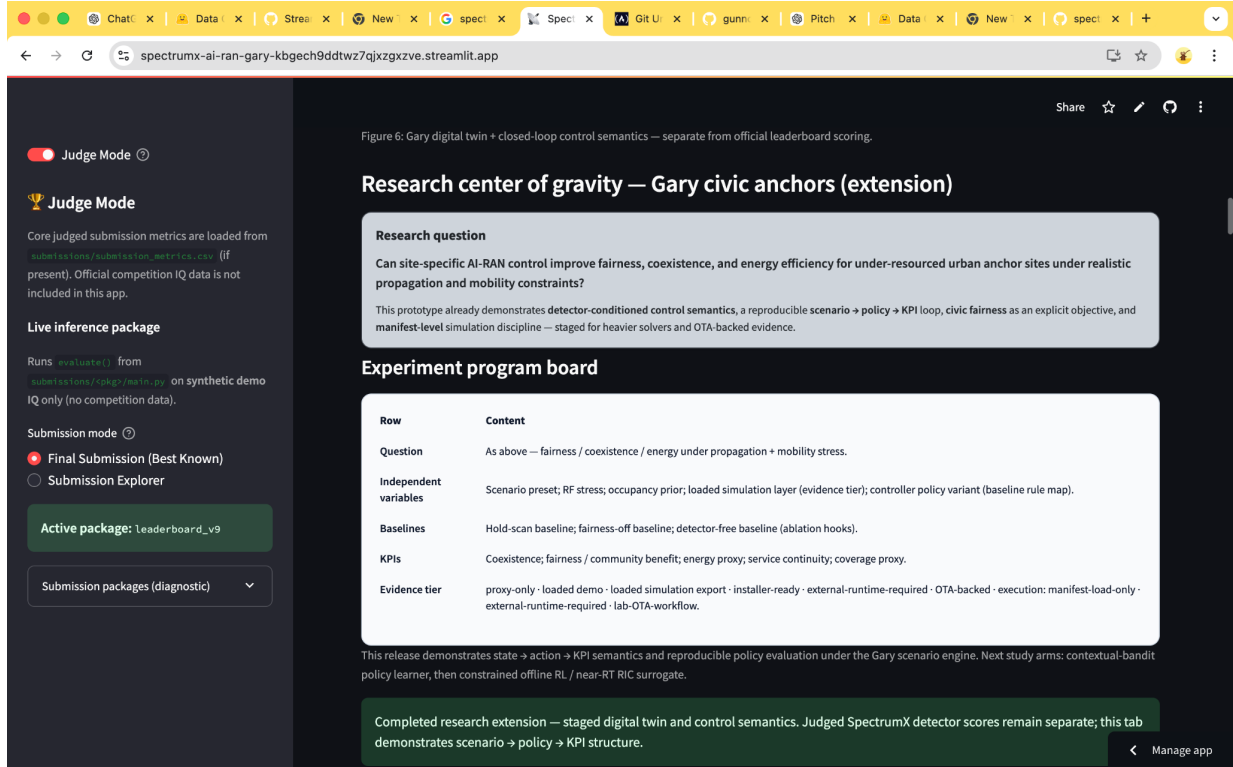
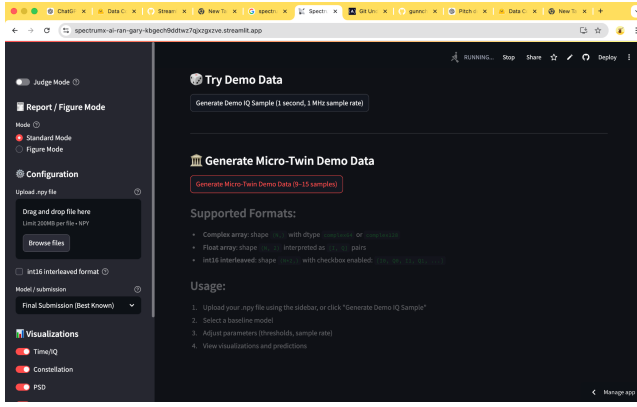
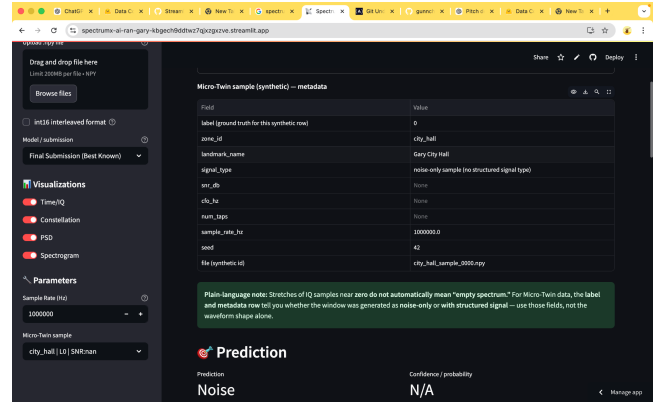


Figure 8: Research center of gravity for the Gary extension. The Streamlit application states the extension research question and the experiment-program board for the three Gary civic anchors, making variables, baselines, KPIs, and evidence tiers explicit.

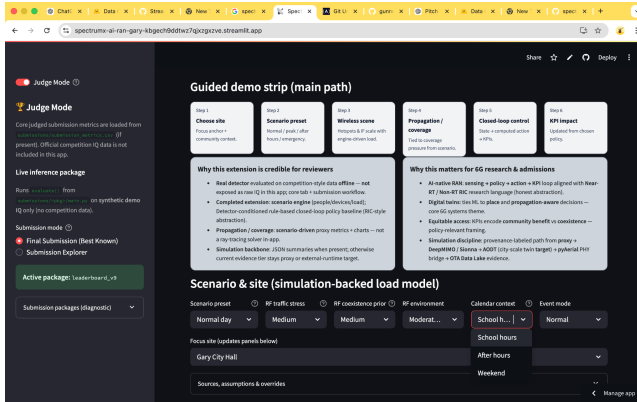


(a) Micro-twin demo generation entry point

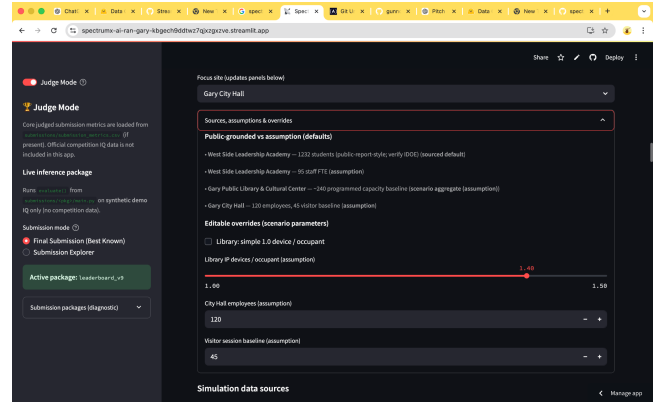


(b) Generated synthetic sample with detector output

Figure 9: Synthetic micro-twin data generation and judged-detector-style inference in the Streamlit app. The left panel shows the report/figure-mode interface for generating demo I/Q samples, while the right panel shows generated sample metadata together with the resulting occupied/noise-style prediction surface.

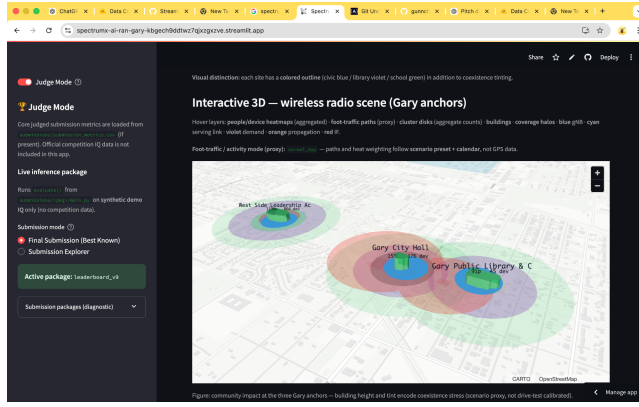


(a) Default extension entry state

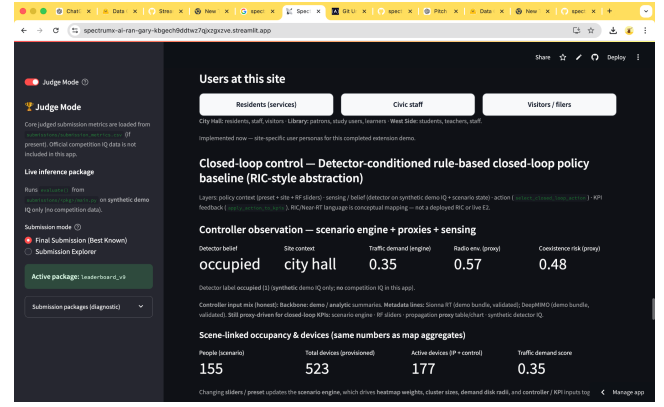


(b) Default sources, assumptions, and overrides

Figure 10: Default digital-twin setup for the Gary extension. The first panel shows the guided-demo strip and initial scenario controls; the second makes sourced defaults, assumptions, and editable overrides explicit rather than presenting all internal values as measured field truth.



(a) Default 3D Gary scene



(b) Default closed-loop observation state

Figure 11: Default scenario walkthrough for the Gary extension. The left panel shows the site-aware scene with the three anchors, while the right panel shows the detector-conditioned rule-based control baseline, its observation state, and the site-linked scenario quantities used as policy inputs.

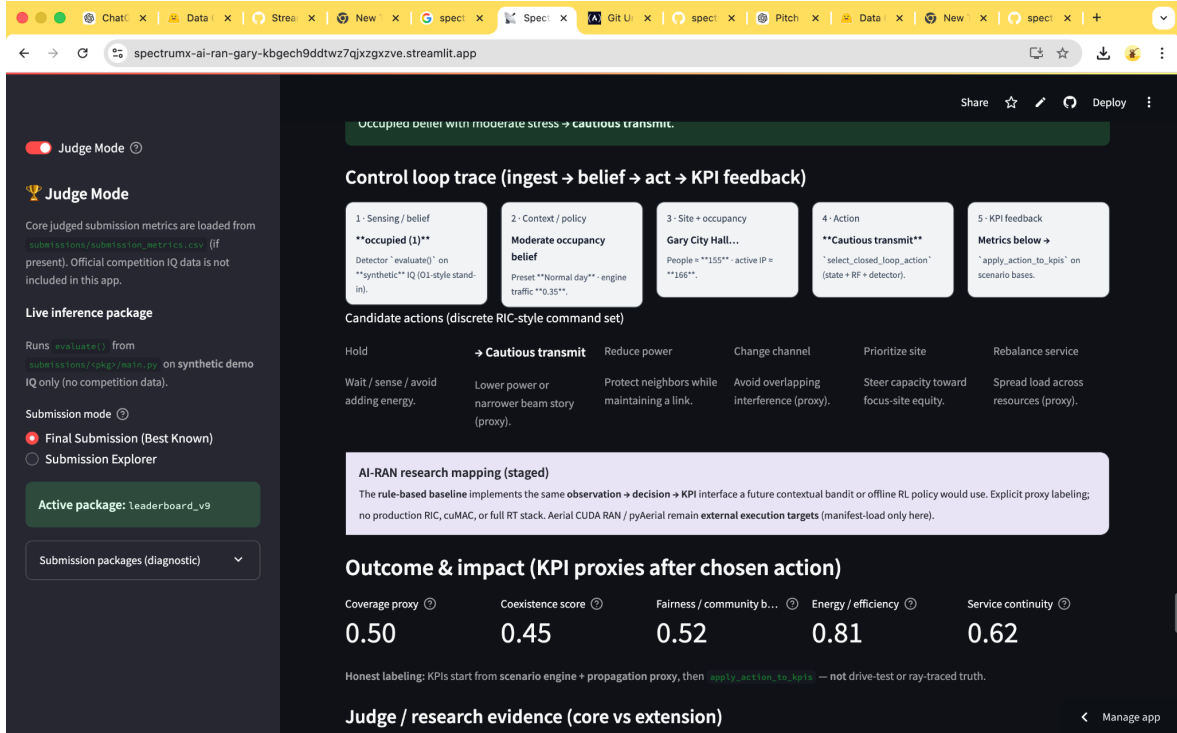
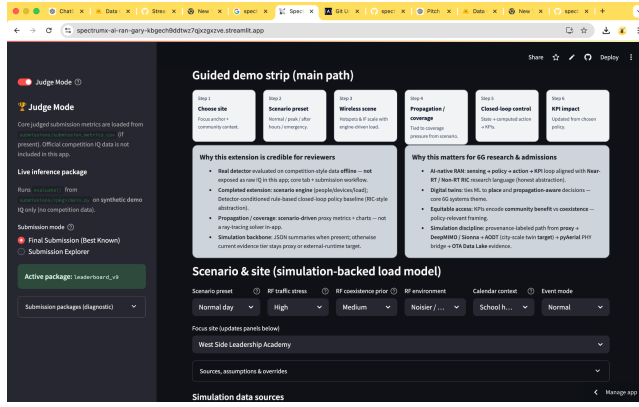
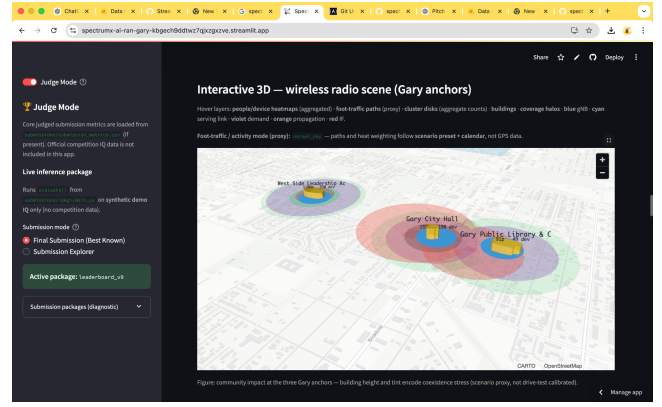


Figure 12: Default KPI state for the extension. This panel shows the control-loop trace, selected action, and proxy KPI outputs under the lower-stress/default scene configuration.

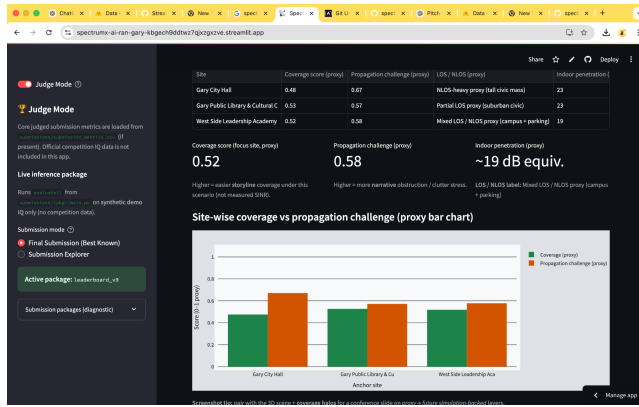


(a) High-RF-stress, noisier-environment settings

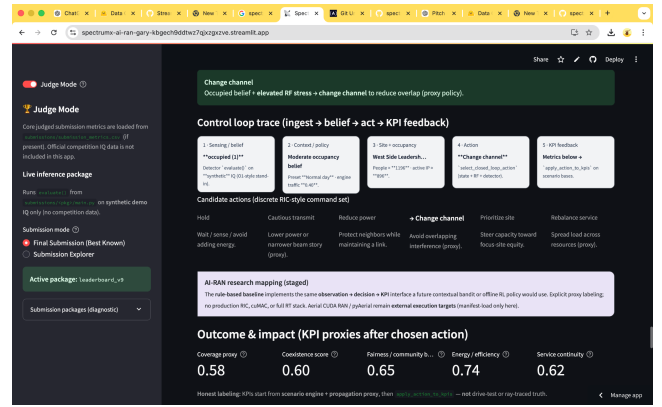


(b) Resulting higher-stress scene

Figure 13: High-RF-stress scenario walkthrough for the extension. The left panel shows the selected school-hours/high-traffic/noisier-environment controls, while the right panel shows the resulting scene under that stress condition.

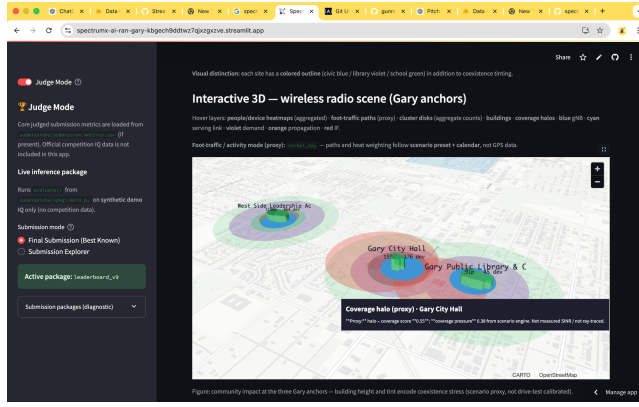


(a) Higher-stress coverage and propagation statistics

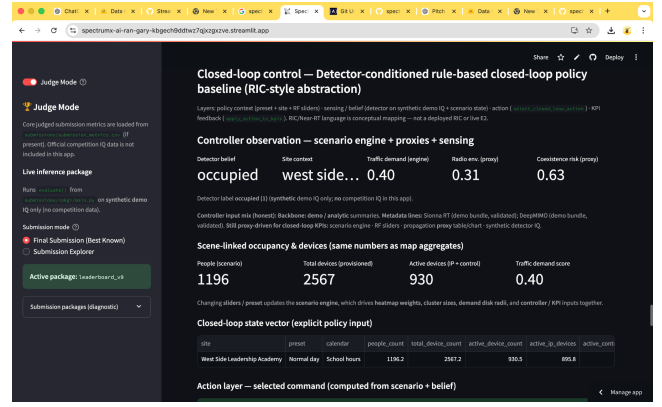


(b) Higher-stress control-loop action and KPI state

Figure 14: Outcome of the high-RF-stress scenario. The left panel summarizes proxy coverage and propagation statistics, while the right panel shows the resulting selected action and KPI proxy outputs under the stressed scene.

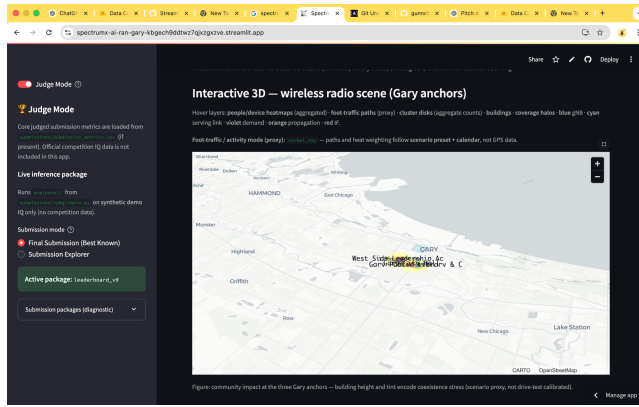


(a) Coverage-halo interaction tooltip

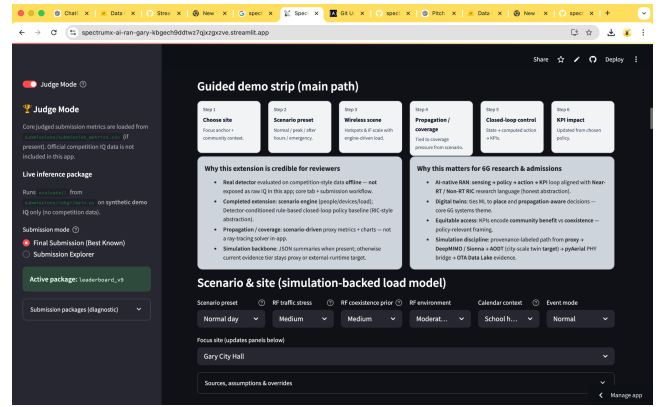


(b) Scenario statistics across anchors

Figure 15: Interactive interpretation aids in the Streamlit extension. The left panel shows tooltip-level semantics for a selected coverage halo, while the right panel shows site-wise scenario statistics used to compare the Gary anchors under the chosen scenario.



(a) Zoomed-out regional context



(b) Guided demo strip with default controls

Figure 16: Additional context for the extension UI. The left panel places the three anchors in a broader regional map view, while the right panel shows the guided-demo strip and the normal-day/default scenario-control surface.

C.2 Current-state UML architecture pack

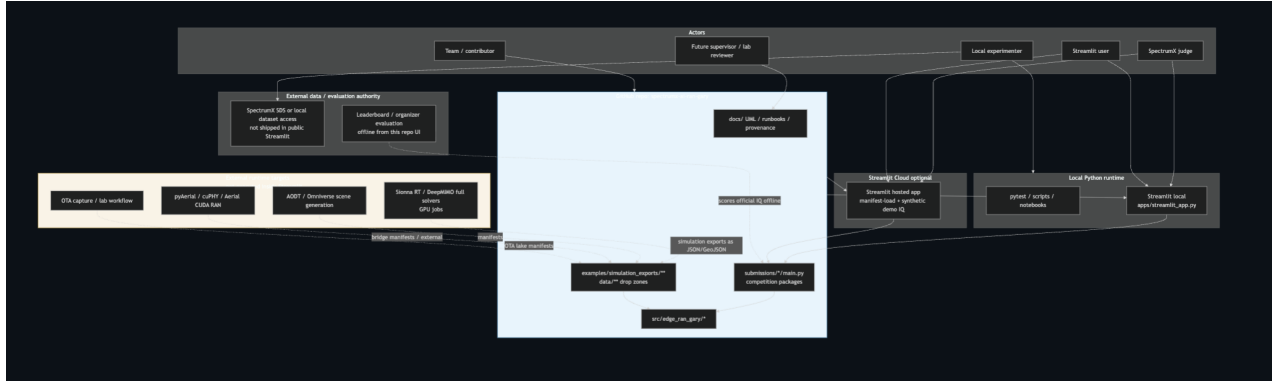


Figure 17: Current-state system context for the judged competition core and completed research extension. The diagram separates external data/evaluation authority, local and cloud execution surfaces, the public repo/documentation layer, and external runtime targets such as AODT, pyAerial, and OTA workflows.

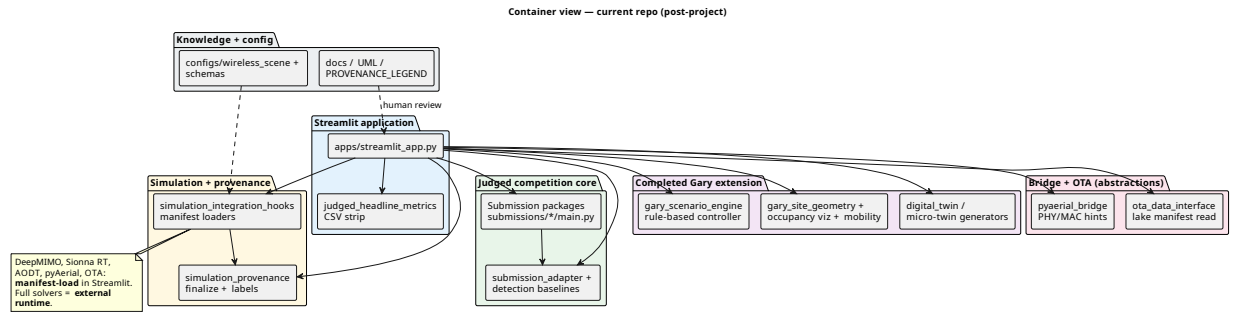


Figure 18: Current-state container view. The main containers are the Streamlit app, judged competition package, feature utilities, baseline detectors, judged headline metrics, and the public/core boundary between the shipped detector package and organizer-side scoring.

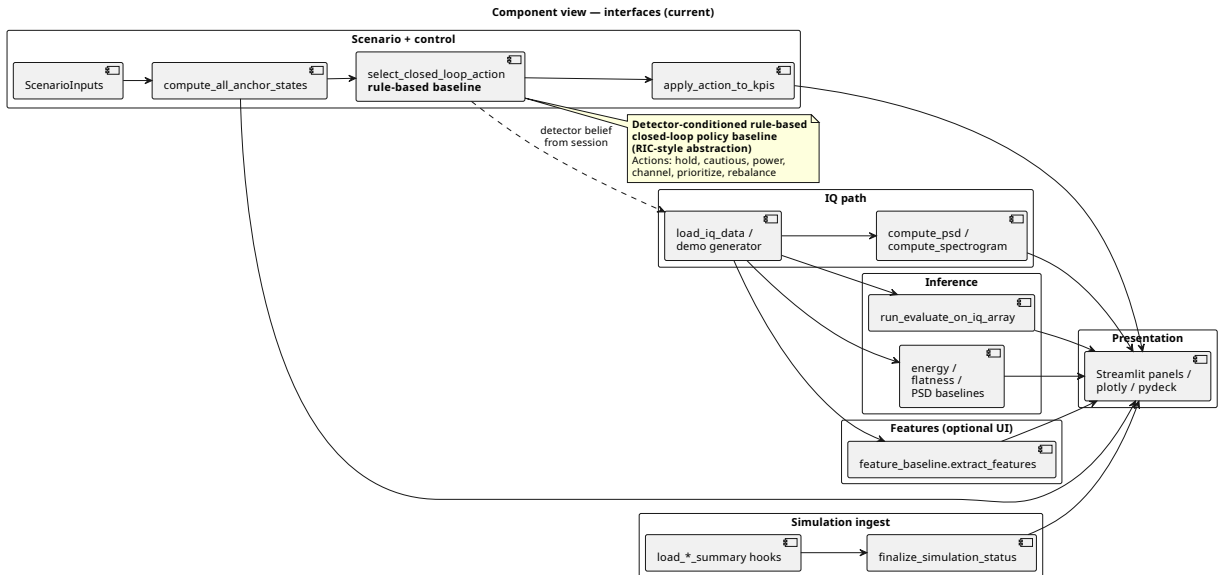


Figure 19: Current-state component view for the Gary extension. The diagram shows the Streamlit extension, scenario inputs, simulation integration hooks, simulation provenance, the rule-based closed-loop controller baseline, site scenario state, and optional pyAerial bridge abstractions.

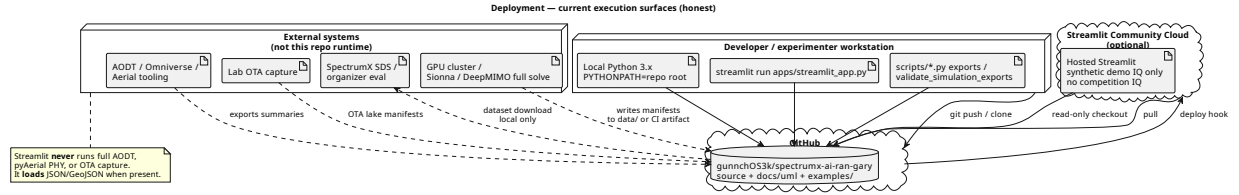


Figure 20: Current-state deployment view. The diagram distinguishes the local Python runtime, the hosted Streamlit surface, the public repo, official scoring performed outside the repo UI, and external simulation or OTA targets that are not part of the shipped Streamlit execution surface.

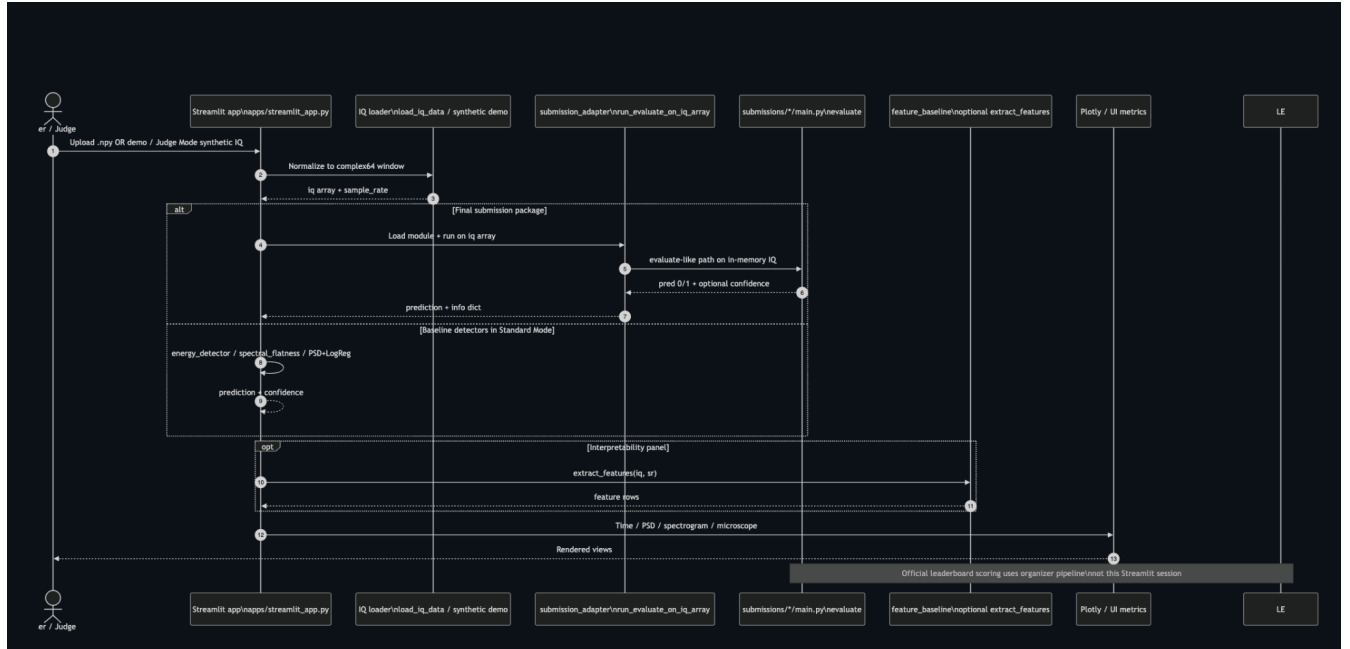


Figure 21: Current competition-inference sequence. A user or judge supplies a sample, the app normalizes the I/Q input, executes the shipped submission path or baseline detectors, and optionally renders feature-level microscope views for interpretability.

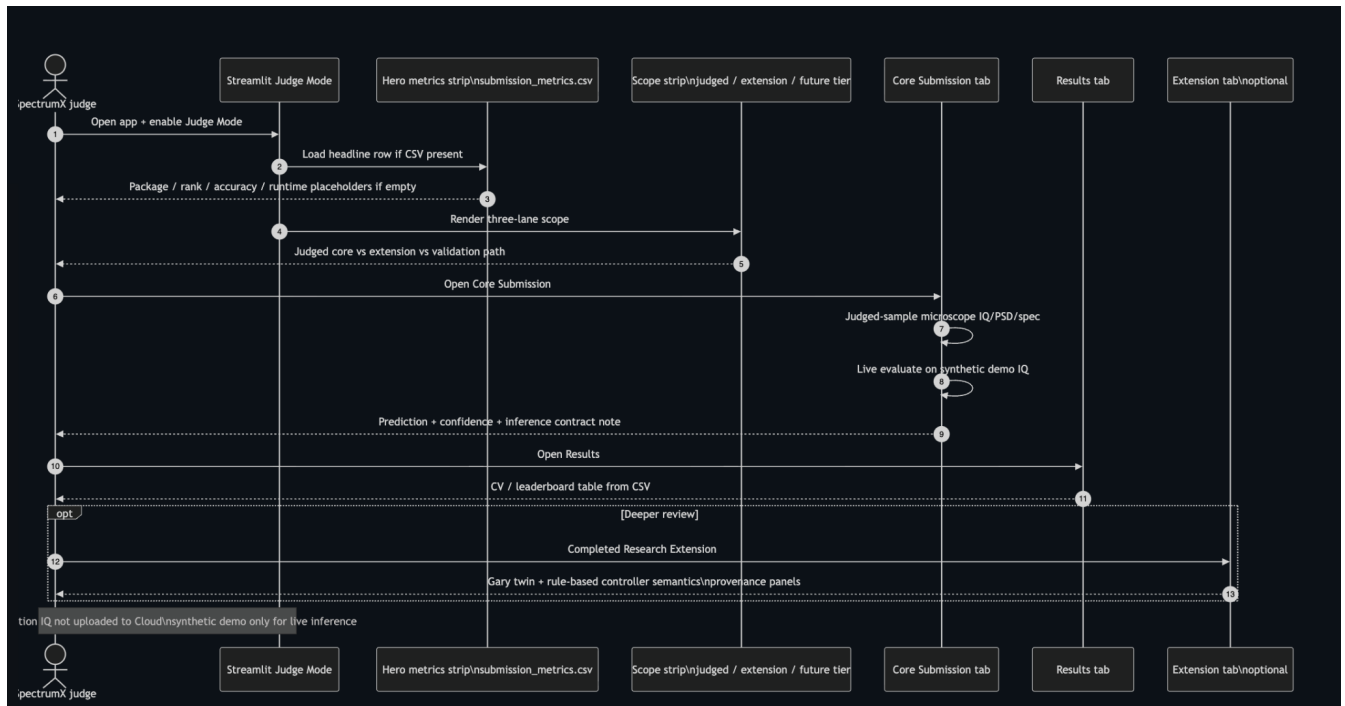


Figure 22: Current judge-review sequence. The flow highlights the headline metrics strip, the judged-versus-extension scope split, the competition-core microscope path, and the optional handoff into the completed research extension panels.

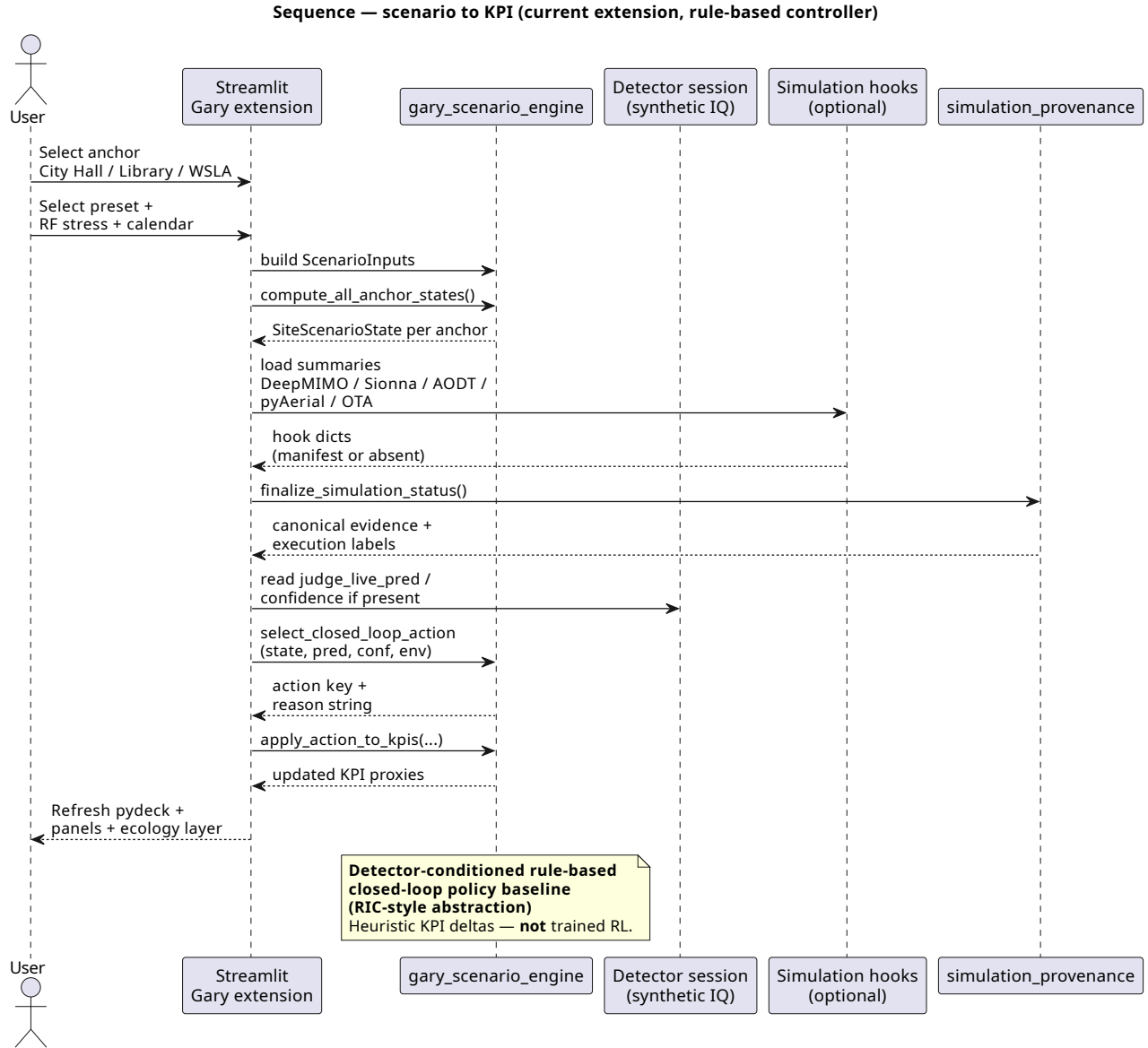


Figure 23: Current extension scenario-to-KPI sequence. A chosen focus site and scenario preset drive the site-state engine, the detector-conditioned rule-based control baseline, and the resulting KPI updates shown in the extension.

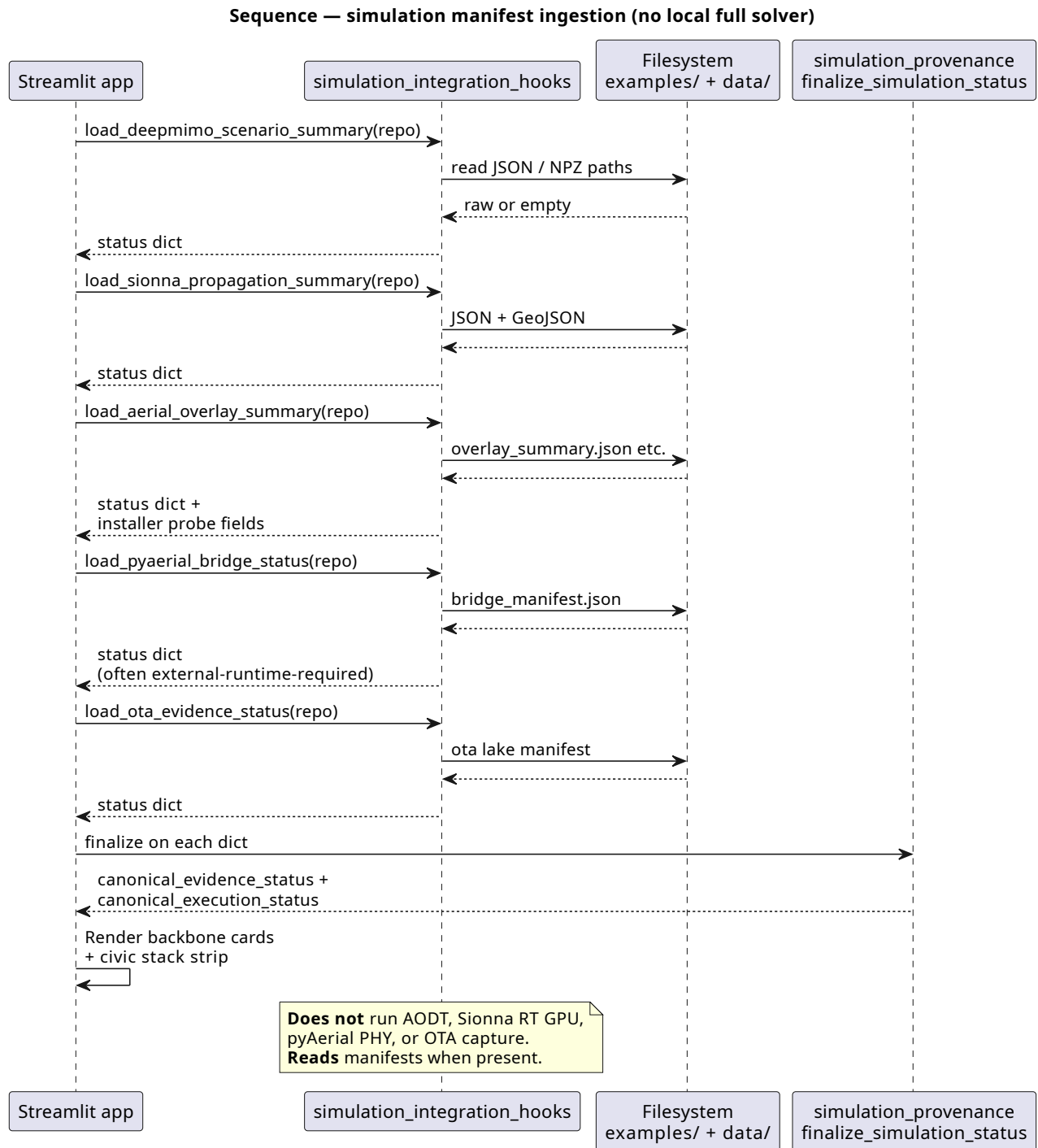


Figure 24: Simulation-manifest ingestion sequence. The extension either merges validated DeepMIMO/Sionna/AODT-style summaries with provenance finalization or falls back to a proxy-only analytic layer before rendering the Streamlit backbone cards and signal-ecology views.

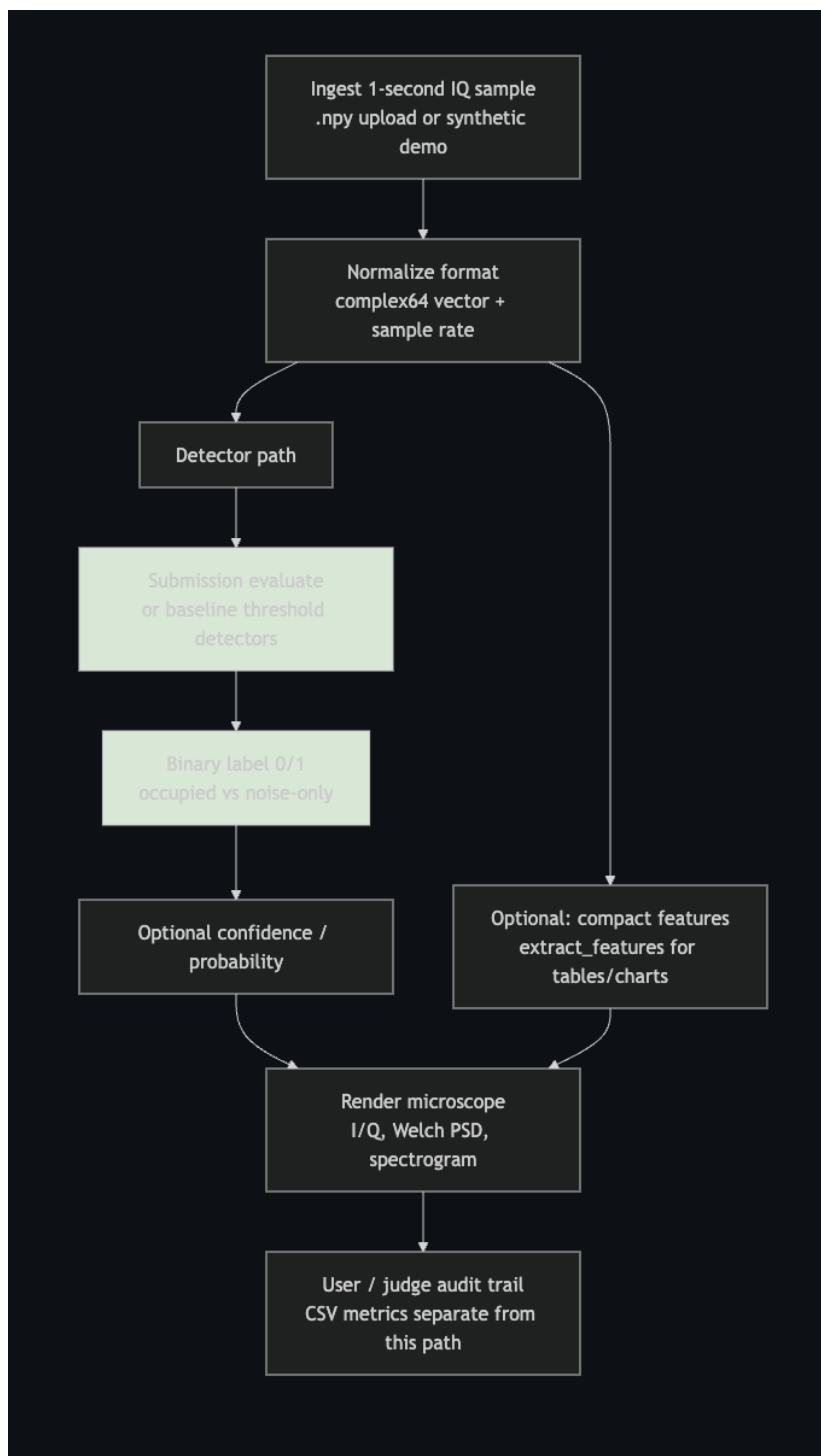
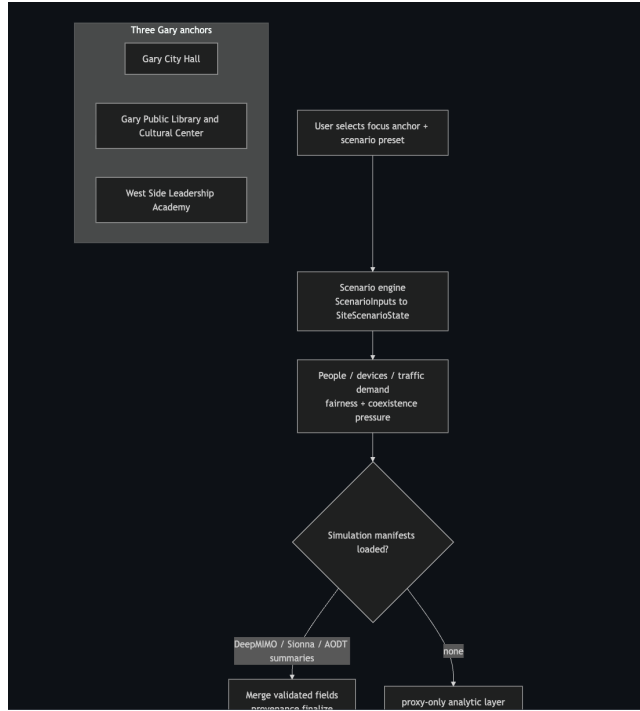
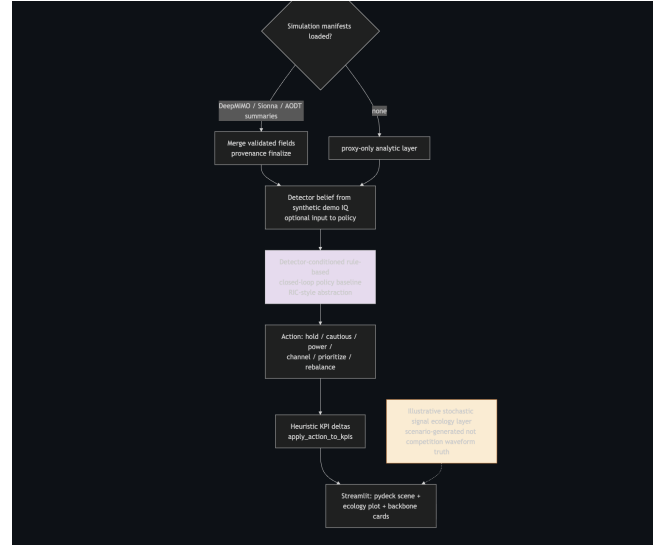


Figure 25: Signal lifecycle activity diagram for the judged detector path. A 1-second IQ sample is ingested, normalized, passed through the competition detector path, optionally summarized by compact features, and then rendered through microscope-style views such as I/Q, Welch PSD, and spectrogram panels.



(a) Upper portion



(b) Lower portion

Figure 26: Signal-ecology activity diagram for the completed research extension. The extension starts from one of the three Gary anchors and a scenario preset, passes through scenario-state construction and simulation-manifest checks, and then feeds the detector-conditioned rule-based policy baseline, heuristic KPI deltas, and the illustrative stochastic signal-ecology layer shown in Streamlit.

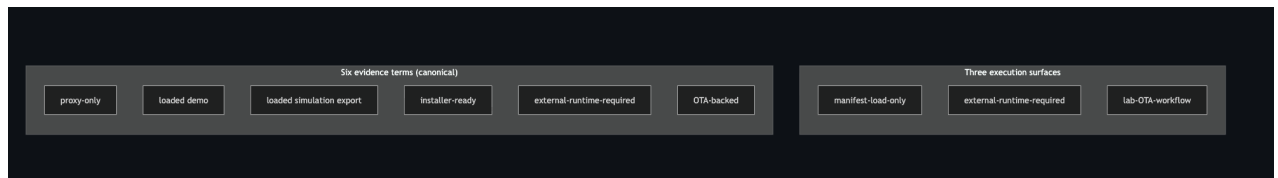


Figure 27: Canonical provenance and execution-surface states used in the extension. The appendix distinguishes proxy-only, loaded demo, loaded simulation export, installer-ready, external-runtime-required, and OTA-backed evidence tiers, together with manifest-load-only, external-runtime-required, and lab-OTA-workflow execution surfaces.

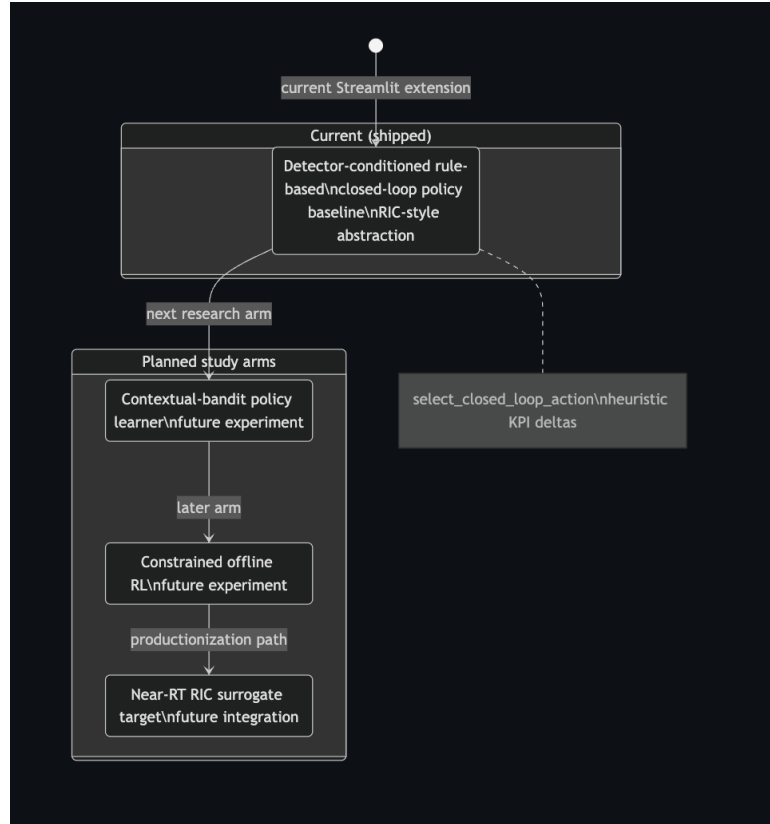


Figure 28: Controller maturity ladder for the Gary extension. The current shipped state is a detector-conditioned rule-based closed-loop policy baseline, while contextual-bandit, constrained offline RL, and a Near-RT RIC surrogate are explicitly future study arms rather than current claims.

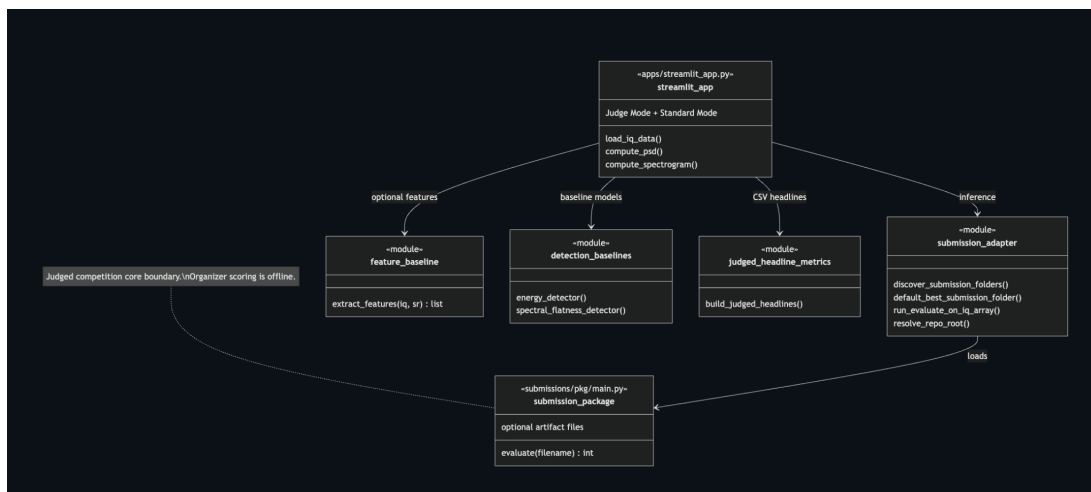


Figure 29: Current class diagram for the judged competition core. The diagram centers the Streamlit judged-core surface, feature extraction module, baseline detector module, judged headline metrics path, and submission adapter that loads the shipped competition package.

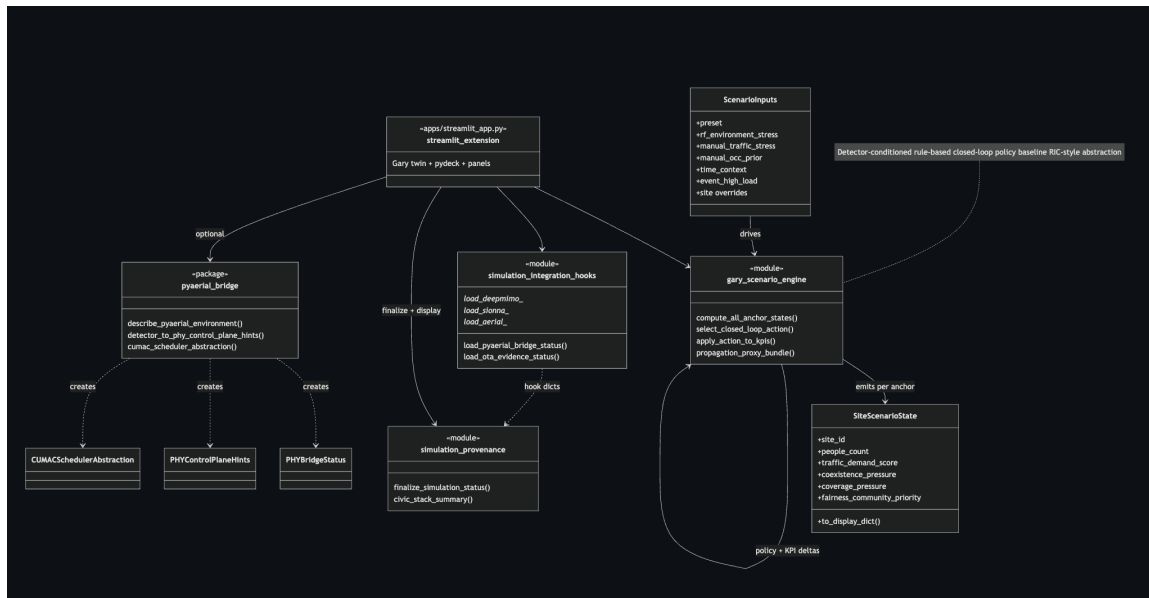


Figure 30: Current class diagram for the extension modules. The extension combines Streamlit presentation, scenario inputs, the Gary scenario engine, site-scenario state objects, simulation hooks, provenance finalization, and optional pyAerial bridge abstractions.

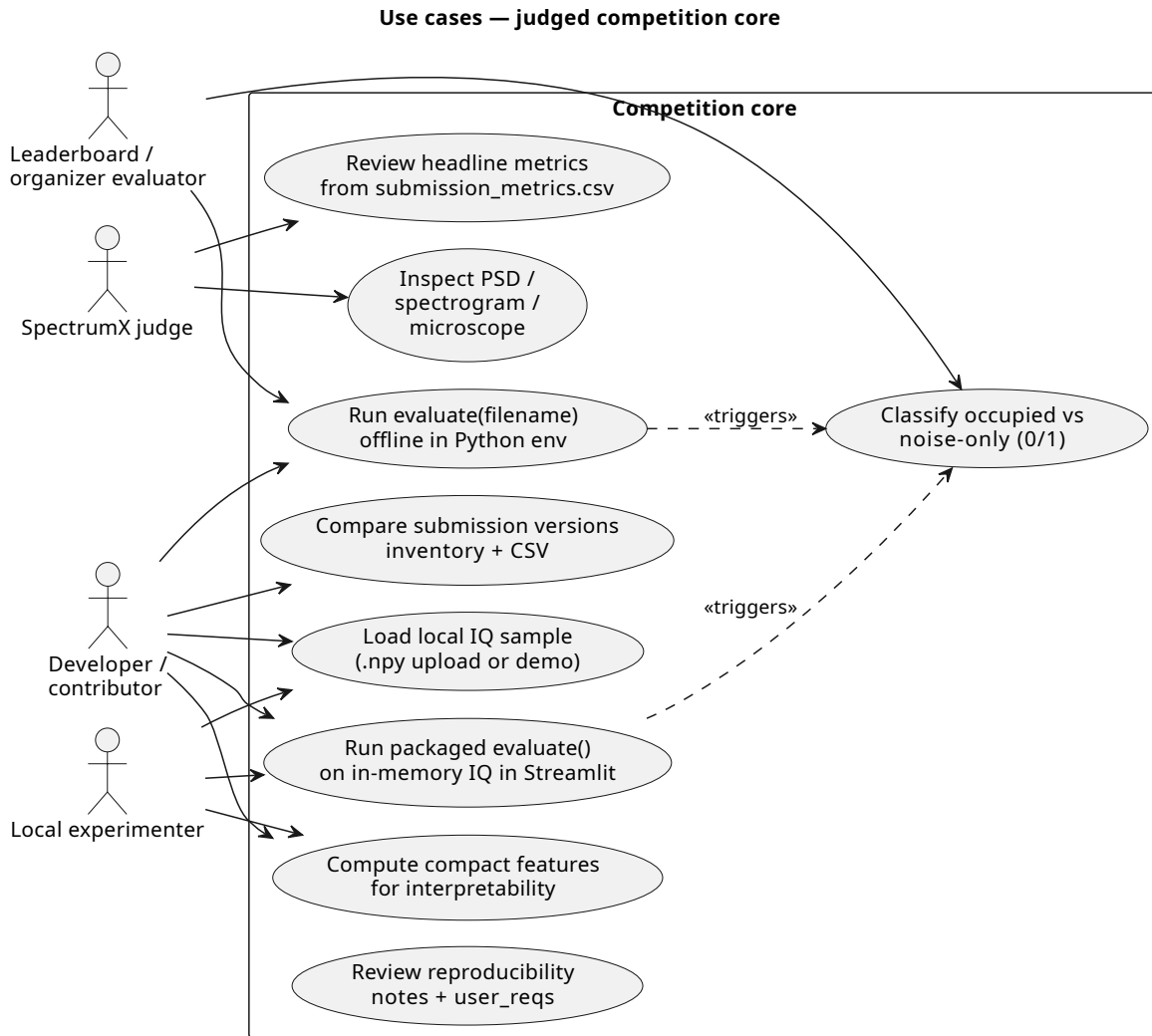


Figure 31: Use-case diagram for the judged competition core. The competition-core surface is organized around local experimentation, detector execution, metric review, and judge-facing inspection of the shipped occupancy-detection path.

Use cases — completed research extension (current)

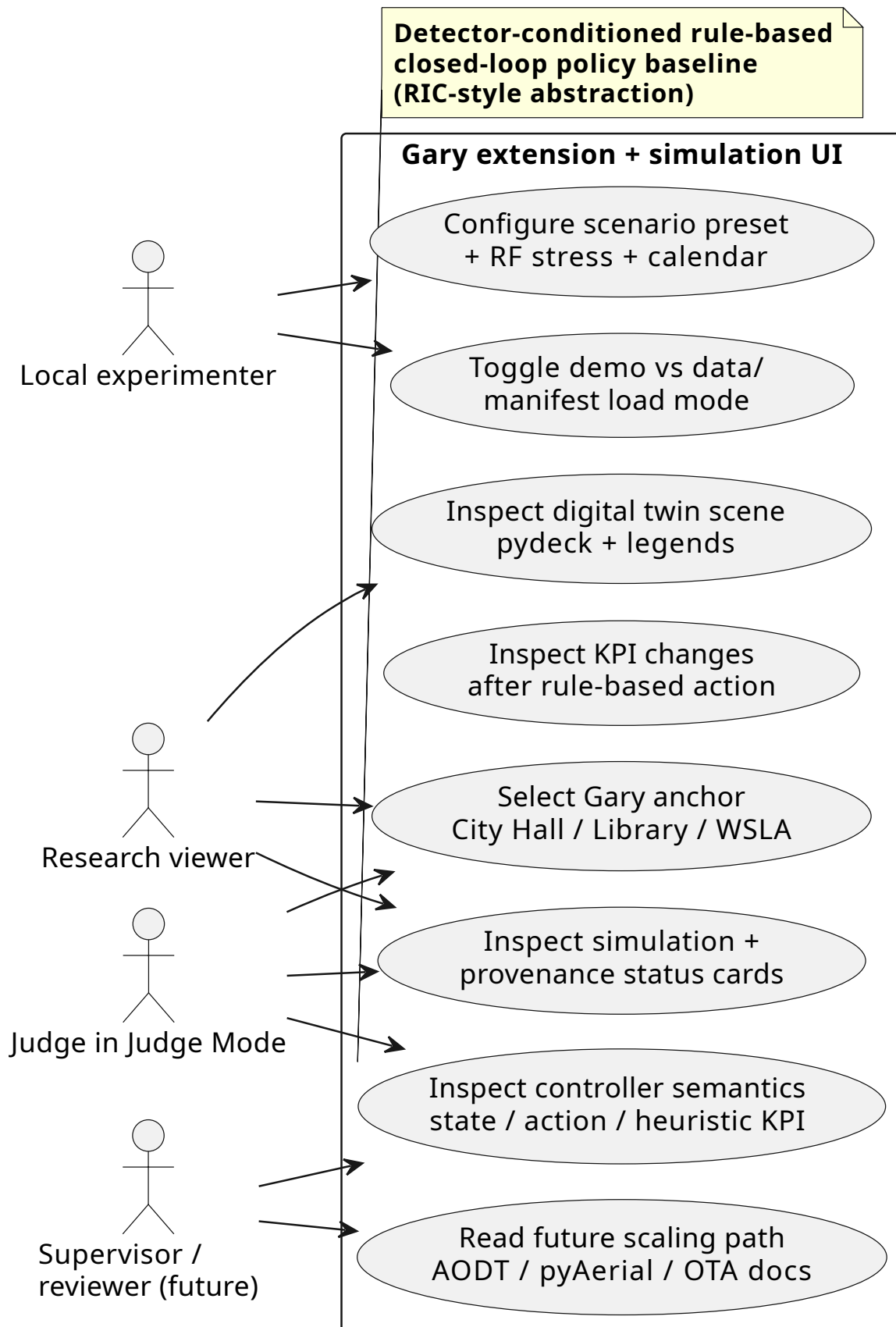


Figure 32: Use-case diagram for the current research extension. The extension adds site selection, scenario configuration, policy/KPI inspection, and review of simulation and provenance status while remaining distinct from the official leaderboard scoring path.

C.3 Role in this submission

These appendix figures should be read as supplementary research documentation rather than as evidence about the scored leaderboard algorithm itself. Their role is to make the broader Gary extension inspectable: they show how the judged detector can sit inside a future site-aware sensing and control workflow, how provenance is tracked honestly, and how the project could scale toward simulation-backed and eventually OTA-backed wireless research while keeping the current shipped scope explicit.